

Լաբորատոր աշխատանք 4

Կառուցվածքային տիպեր C++ լեզվում · «Ավտոմեքենաների տվյալներ»

Կատարող՝ Hakob Avagyan · 2026

Աշխատանքի նպատակ

Ամրապնդել `struct` կառուցվածքների, կառուցվածքների զանգվածների և ցուցիչների հետ աշխատանքը, ինչպես նաև ֆունկցիաներին պարամետրեր փոխանցելու եղանակները (`const` հղում, հղում, ցուցիչ)՝ պահպանելով մուտքային տվյալների ստուգման պահանջները:

Խնդրի պահանջը

Ստեղծել `Car` կառուցվածք՝ `brand`, `model`, `color` և `price` դաշտերով: Մուտքագրել 8 ավտոմեքենայի տվյալներ: Ֆունկցիաների միջոցով՝

- արտածել բոլոր ավտոմեքենաները,
- գտնել ամենաթանկ ավտոմեքենան,
- արտածել տրված մակնիշի ավտոմեքենաները,
- դասավորել ավտոմեքենաները գնի աճման կարգով:

Ընդհանուր պահանջներ

- Առանձին `struct` տիպ՝ դաշտերի իմաստալից անգլերեն անուններով:
- Մուտքագրումը, որոնումը և դասակարգումը՝ առանձին ֆունկցիաներով:
- Միայն ընթերցող ֆունկցիաներում `const` հղում կամ `const` զանգված, փոփոխողներում հղում կամ ցուցիչ:
- Մուտքային տվյալների թույլատրելի սահմանների ստուգում:
- Դասակարգելիս փոխանակել ամբողջ կառուցվածքային տարրը:
- Ընթերցման վերնագրերով արտածում և կոդի կարճ մեկնաբանություններ:

Տեսական մաս

Կառուցվածք (`struct`)՝ օգտագործողի կողմից սահմանված տիպ, որը մեկ անվան տակ միավորում է տարբեր տիպերի դաշտեր: `Car`-ում երեք դաշտ ունեն `string` տիպ (մակնիշ, մոդել, գույն), իսկ գինը՝ `double`: Դաշտին դիմում են կետով (`car.price`), իսկ ցուցիչի միջոցով՝ սլաքով (`ptr->price`): Կառուցվածքների զանգվածը (`Car cars[8]`) թույլ է տալիս մշակել նույնատիպ գրառումների խումբ, իսկ կառուցվածքների վերագրումը (`cars[j] = cars[j + 1]`) պատճենում է բոլոր դաշտերը միանգամից, ներառյալ տողերը:

Պարամետրերի փոխանցում. `const Car&` — առանց պատճենման և փոփոխման իրավունքի, `Car&` — օրիգինալը փոփոխելու համար, `const Car cars[]` — զանգված միայն ընթերցման համար: Ամենաթանկ մեքենան որոնող ֆունկցիան վերադարձնում է ցուցիչ զանգվածի տարրին, իսկ դատարկ զանգվածի դեպքում՝ `nullptr`, որը ստուգվում է օգտագործելուց առաջ: Մակնիշով որոնումը կատարվում է առանց մեծատառ/փոքրատառ տարբերության (`toyota = Toyota`):

Ծրագրի ֆունկցիաները

Ֆունկցիա	Պարամետրեր	Նշանակություն
readCar	Car&, int	մեկ գրառման մուտքագրում և ստուգում (տեքստային դաշտեր 1-30 նիշ, գին 500-5 000 000 \$)
inputCars	Car[]	8 ավտոմեքենայի մուտքագրում
printCars	const Car[]	բոլոր ավտոմեքենաների աղյուսակային արտածում վերնագրով
printCarRow	const Car&	մեկ մեքենայի տողի արտածում
findMostExpensive	const Car[]	որոնում. վերադարձնում է ցուցիչ ամենաթանկ մեքենային կամ nullptr
printCarsByBrand	const Car[], const string&	տրված մակնիշի մեքենաների արտածում, վերադարձնում է գտնվածների քանակը
sortByPriceAscending	Car[]	պղպջակային դասակարգում ըստ գնի, փոխանակվում է ամբողջ Car -ը

Ծրագրի կոդը

cars.cpp

```

1 // Լաբորատոր աշխատանք 4 – Ավտոմեքենաների տվյալներ
2 #include <iostream>
3 #include <iomanip>
4 #include <string>
5 #include <limits>
6 #include <cctype>
7 #include <cstdlib>
8 #ifdef _WIN32
9 #include <windows.h>
10 #endif
11 using namespace std;
12
13 const int CAR_COUNT = 8;
14
15 // Թույլատրելի սահմաններ
16 const double MIN_PRICE = 500, MAX_PRICE = 5000000; // $
17 const size_t MAX_TEXT_LENGTH = 30; // տողային դաշտի առավելագույն երկարություն
18
19 struct Car {
20     string brand; // մակնիշ
21     string model; // մոդել
22     string color; // գույն
23     double price; // գին, $
24 };
25
26 // UTF-8 տողի արտածում տրված լայնությամբ (հայերեն տառերը 2 բայթ են)
27 void printCell(const string& text, int width) {
28     int symbols = 0;
29     for (unsigned char c : text)
30         if ((c & 0xC0) != 0x80) symbols++;
31     cout << text;
32     for (int i = symbols; i < width; i++) cout << ' ';
33 }
34
35 // Տառերի համեմատում առանց մեծատառ/փոքրատառ տարբերության (լատինատառ)
36 bool equalsIgnoreCase(const string& a, const string& b) {
37     if (a.size() != b.size()) return false;
38     for (size_t i = 0; i < a.size(); i++)
39         if (tolower((unsigned char)a[i]) != tolower((unsigned char)b[i])) return false;

```

```

40     return true;
41 }
42
43 // Տողային դաշտի ստուգում՝ դատարկ չէ և չափազանց երկար չէ
44 bool isValidText(const string& text) {
45     return !text.empty() && text.size() <= MAX_TEXT_LENGTH;
46 }
47
48 // Մեկ ավտոմեքենայի մուտքագրում ստուգումներով (փոփոխող՝ հղումով)
49 void readCar(Car& car, int index) {
50     while (true) {
51         cout << "Մեքենա " << index + 1 << " (մականիշ մոդել գույն գին): ";
52         if (!(cin >> car.brand >> car.model >> car.color >> car.price)) {
53             if (cin.eof()) { cout << "\nՄուտքն ավարտվեց անսպասելի:\n"; exit(1); }
54             cin.clear();
55             cin.ignore(numeric_limits<streamsize>::max(), '\n');
56             cout << " Սխալ: գինը պետք է լինի թիվ:\n";
57             continue;
58         }
59         if (!isValidText(car.brand) || !isValidText(car.model) || !isValidText(car.color))
60             cout << " Սխալ: տեքստային դաշտերը պետք է լինեն 1-30 նիշ:\n";
61         else if (car.price < MIN_PRICE || car.price > MAX_PRICE)
62             cout << " Սխալ: գինը պետք է լինի 500-5 000 000 $:\n";
63         else
64             return; // բոլոր դաշտերը ճիշտ են
65     }
66 }
67
68 // Ամբողջ զանգվածի մուտքագրում
69 void inputCars(Car cars[], int count) {
70     for (int i = 0; i < count; i++)
71         readCar(cars[i], i);
72 }
73
74 // Մեկ տողի արտածում (միայն ընթերցող՝ const հղում)
75 void printCarRow(const Car& car) {
76     printCell(car.brand, 12); printCell(car.model, 12); printCell(car.color, 10);
77     cout << right << setw(10) << fixed << setprecision(0) << car.price << left << '\n';
78 }
79
80 // Աղյուսակի վերնագիր
81 void printHeader(const string& title) {
82     cout << "\n=== " << title << " ===\n";
83     printCell("Մականիշ", 12); printCell("Մոդել", 12); printCell("Գույն", 10);
84     printCell("Գին, $", 10); cout << '\n';
85     cout << string(44, '-') << '\n';
86 }
87
88 // Բոլոր ավտոմեքենաների արտածում
89 void printCars(const Car cars[], int count, const string& title) {
90     printHeader(title);
91     for (int i = 0; i < count; i++)
92         printCarRow(cars[i]);
93 }
94
95 // Ամենաթանկ ավտոմեքենան (դատարկ զանգվածի դեպքում՝ nullptr)
96 const Car* findMostExpensive(const Car cars[], int count) {
97     if (count <= 0) return nullptr;
98     const Car* best = &cars[0];
99     for (int i = 1; i < count; i++)
100         if (cars[i].price > best->price)
101             best = &cars[i];
102     return best;
103 }
104
105 // Տրված մականիշի ավտոմեքենաների արտածում, վերադարձնում է գտնվածների քանակը
106 int printCarsByBrand(const Car cars[], int count, const string& brand) {
107     printHeader("<< " + brand + ">> մականիշի ավտոմեքենաներ");
108     int found = 0;

```

```

109     for (int i = 0; i < count; i++)
110         if (equalsIgnoreCase(cars[i].brand, brand)) {
111             printCarRow(cars[i]);
112             found++;
113         }
114     if (found == 0) cout << "Այդ մակնիշի ավտոմեքենա չի գտնվել:\n";
115     return found;
116 }
117
118 // Դասակարգում ըստ գնի աճման (փոխանակվում է ամբողջ կառուցվածքը)
119 void sortByPriceAscending(Car cars[], int count) {
120     for (int i = 0; i < count - 1; i++)
121         for (int j = 0; j < count - 1 - i; j++)
122             if (cars[j].price > cars[j + 1].price) {
123                 Car temp = cars[j];
124                 cars[j] = cars[j + 1];
125                 cars[j + 1] = temp;
126             }
127 }
128
129 // Որոնվող մակնիշի մուտքագրում
130 string readBrand() {
131     string brand;
132     while (true) {
133         cout << "\nՈրոնվող մակնիշը: ";
134         if (!(cin >> brand)) exit(1);
135         if (isValidText(brand)) return brand;
136         cout << " Սխալ: մակնիշը պետք է լինի 1-30 նիշ:\n";
137     }
138 }
139
140 int main() {
141     #ifdef _WIN32
142         SetConsoleOutputCP(CP_UTF8);
143         SetConsoleCP(CP_UTF8);
144     #endif
145     Car cars[CAR_COUNT];
146
147     cout << "Մուտքագրեք " << CAR_COUNT << " ավտոմեքենայի տվյալները\n";
148     inputCars(cars, CAR_COUNT);
149     printCars(cars, CAR_COUNT, "Մուտքագրված ավտոմեքենաներ");
150
151     const Car* top = findMostExpensive(cars, CAR_COUNT);
152     cout << "\n=== Ամենաթանկ ավտոմեքենան ===\n";
153     if (top != nullptr)
154         cout << top->brand << " " << top->model << ", " << top->color
155             << ", գին " << fixed << setprecision(0) << top->price << " $\n";
156
157     string brand = readBrand();
158     printCarsByBrand(cars, CAR_COUNT, brand);
159
160     sortByPriceAscending(cars, CAR_COUNT);
161     printCars(cars, CAR_COUNT, "Դասակարգված ըստ գնի աճման");
162     return 0;
163 }

```

Օրինակային մուտք և ելք

Մուտքում միտումնաւոր ներառված են սխալ արժեքներ (բացասական գին, տառեր թվի փոխարեն)՝ ստուգումների աշխատանքը ցույց տալու համար: Կանաչով նշված է օգտատիրոջ մուտքը:

Terminal — g++ -std=c++17 -Wall cars.cpp -o cars && ./cars

```

Մուտքագրեք 8 ավտոմեքենայի տվյալները
Մեքենա 1 (մակնիշ մոդել գույն գին): Toyota Camry White 28000
Մեքենա 2 (մակնիշ մոդել գույն գին): BMW X5 Black 65000
Մեքենա 3 (մակնիշ մոդել գույն գին): Mercedes E200 Silver -100

```

Սխալ : գինը պետք է լինի 500–5 000 000 \$:

Մեքենա 3 (մակնիշ մոդել գույն գին): Mercedes E200 Silver 52000

Մեքենա 4 (մակնիշ մոդել գույն գին): Toyota RAV4 Red abc

Սխալ : գինը պետք է լինի թիվ:

Մեքենա 4 (մակնիշ մոդել գույն գին): Toyota RAV4 Red 34000

Մեքենա 5 (մակնիշ մոդել գույն գին): Kia Rio Blue 15500

Մեքենա 6 (մակնիշ մոդել գույն գին): BMW 320i Gray 41000

Մեքենա 7 (մակնիշ մոդել գույն գին): Hyundai Tucson Green 31000

Մեքենա 8 (մակնիշ մոդել գույն գին): Lada Vesta White 12000

=== Մուտքագրված ավտոմեքենաներ ===

Մակնիշ	Մոդել	Գույն	Գին, \$
Toyota	Camry	White	28000
BMW	X5	Black	65000
Mercedes	E200	Silver	52000
Toyota	RAV4	Red	34000
Kia	Rio	Blue	15500
BMW	320i	Gray	41000
Hyundai	Tucson	Green	31000
Lada	Vesta	White	12000

=== Ամենաթանկ ավտոմեքենան ===

BMW X5, Black, գին 65000 \$

Որոնվող մակնիշը: **toyota**

=== «toyota» մակնիշի ավտոմեքենաներ ===

Մակնիշ	Մոդել	Գույն	Գին, \$
Toyota	Camry	White	28000
Toyota	RAV4	Red	34000

=== Դասակարգված ըստ գնի աճման ===

Մակնիշ	Մոդել	Գույն	Գին, \$
Lada	Vesta	White	12000
Kia	Rio	Blue	15500
Toyota	Camry	White	28000
Hyundai	Tucson	Green	31000
Toyota	RAV4	Red	34000
BMW	320i	Gray	41000
Mercedes	E200	Silver	52000
BMW	X5	Black	65000

Հանձնման ստուգաթերթ

Կետ	Արդյունք
Ծրագիրը կոմպիլացվում և աշխատում է առանց սխալների	✓ g++ -Wall -Wextra, 0 զգուշացում
Բոլոր մուտքային սահմանները ստուգված են	✓ տեքստային դաշտերի երկարություն, գին, թվային ձևաչափ
Ֆունկցիաների պարամետրերը ընտրված են ըստ նշանակության	✓ const` ընթերցման, հղում/գանգված` փոփոխման
Դինամիկ հիշողությունը ազատված է	✓ դինամիկ հիշողություն չի կիրառվել
Օրինակային մուտքն ու ելքը կցված են	✓ տես վերևում

Եզրակացություն

Ստեղծվեց Car կառուցվածք և 8 տարրից բաղկացած կառուցվածքների զանգված: Ծրագիրը ստուգումներով մոտքագրում է տվյալները, արտածում է բոլոր ավտոմեքենաները, գտնում է ամենաթանկը (օրինակում BMW X5, 65 000 \$), արտածում է տրված մակնիշի մեքենաները (օրինակում Toyota-ի 2 մեքենա) և դասակարգում է դրանք ըստ գնի աճման՝ փոխանակելով ամբողջ կառուցվածքը: